

HW2 - Traditional vs. Function-as-a-Service: A Ride-Sharing Dispatch System

Overview & System Design

We built a ride-sharing / fleet dispatch system (like a small Uber/Lyft) twice: once as a traditional monolith and once as a Function-as-a-Service design. We picked this scenario over the hospital/hotel/airport/university examples in the assignment because it has several interacting, stateful resources (drivers, vehicles, zones, rides) with real contention between them, which gives the feature-extension and performance comparisons actual substance instead of being a CRUD exercise.

A ride moves through a lifecycle (REQUESTED to MATCHED to IN_PROGRESS to COMPLETED, or CANCELLED), driven by 12 operations: `request_ride`, `match_driver_to_rider`, `track_vehicle_location`, `start_trip`, `complete_trip`, `calculate_fare`, `process_surge_pricing`, `handle_cancellation`, `manage_driver_shift`, `process_payment`, `rate_and_review`, `schedule_vehicle_maintenance`.

Traditional ([Traditional/ride_system.py](#)) is one `RideSharingSystem` class. All 12 operations are methods that read and mutate the same in-memory Python objects directly: one process, shared memory, direct method calls, no serialization between operations.

FaaS ([FaaS/functions/](#)) uses one file per operation, each with a `handler(state, ...)` function and a standalone CLI entry point. Run as a script, each function is a genuinely separate OS process that loads the *entire* shared state from a JSON file ([FaaS/state_store.py](#)), applies its logic, and writes the whole state back. There is deliberately no file locking on that shared state, so two functions racing to update it can clobber each other, a real property of naive FaaS-over-shared-storage. The fare formula and other business constants are reimplemented independently on the FaaS side rather than imported from Traditional/, so the two architectures don't secretly share code.

Both implementations are verified against each other directly. `tests/test_scenarios.py` runs identical workflows on both and asserts equivalent results, and both `run_workload.py` drivers process the same deterministic 200-ride workload and produce byte-identical output (completed=200 total_fare=2028.00). 47 automated tests pass across both architectures.

Part 3 - Feature Extension: Automatic Driver Reassignment

We extended both systems with one feature: when a driver cancels mid-ride, automatically find a replacement driver instead of leaving the rider with a CANCELLED ride. We picked this because it isn't a new isolated operation. It *composes* two operations that already existed (freeing a driver, re-running matching), which is exactly the kind of change that exposes real architectural differences.

Traditional: trivial. One new ~20-line method, `handle_driver_cancellation_with_reassignment`, frees the old driver, reopens the ride, and calls `self.match_driver_to_rider(...)` again. One file touched, no new concepts, because matching is just another method on the same shared object.

FaaS: required a brand-new file, [reassign_ride.py](#), an orchestrator. This is a category of function that didn't exist among the original 12. Every other FaaS function is independent by design (no function calls another), but this feature can't be expressed that way, so `reassign_ride.py` has to shell out to `match_driver_to_rider.py` as a subprocess, quietly breaking the "minimal dependencies between functions" principle the rest of the system followed. We also had to add an `exclude_driver_id` parameter to `match_driver_to_rider` in *both* architectures, so a freshly-freed driver isn't immediately re-offered the ride they just abandoned (a bug our own tests caught before it shipped).

Risk: the FaaS version is also structurally riskier. Its two steps (free driver / reopen ride, then re-match) are two separate process invocations writing to disk in between. If the second step fails or the machine dies mid-sequence, the ride is left REQUESTED with no driver and nothing retrying it. That partial-failure state cannot happen in the Traditional version, where both steps run under one Python call stack.

Conclusion: Traditional is decisively easier to extend for this feature. FaaS stays cheap to extend only while new features fit inside one function's boundary. The moment a feature needs *sequencing* across previously-independent triggers, it either needs new coordination infrastructure (a queue, an event bus) or has to compromise the independence that was the point of choosing FaaS.

Part 4 - Performance Evaluation

Methodology. Both architectures ran the identical deterministic 200-ride workload (1,200 total operations), producing the exact same result, so the comparison is apples-to-apples on functionally identical work. We used `perf stat -r 5` for OS-level counters and `py-spy record --subprocesses` for flamegraphs. `perf` record's frame-pointer call-graph unwinding did not reliably capture Python stacks under our KVM guest's nested virtualization, so `py-spy` (which samples the CPython interpreter's frame stack directly) was used for the flamegraph portion only; the `perf stat` numbers below are unaffected by that.

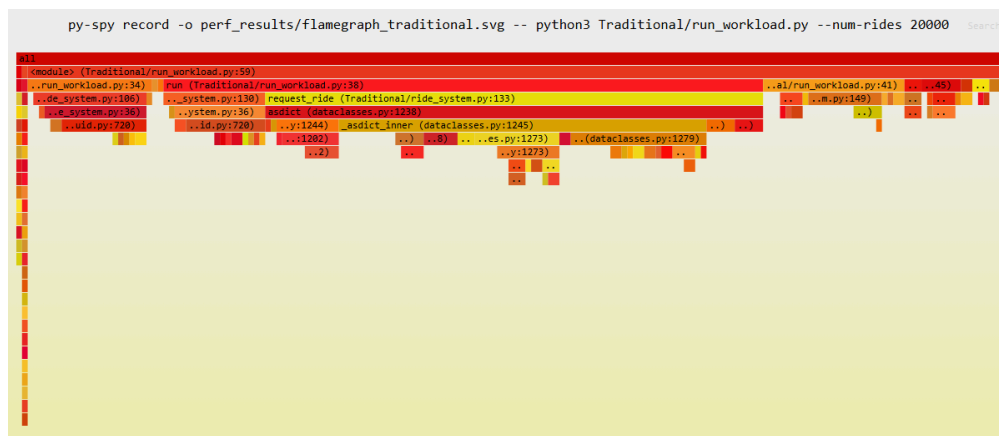
Metric	Traditional	FaaS	FaaS / Traditional
Wall time	0.0790 s	78.294 s	~992×

Metric	Traditional	FaaS	FaaS / Traditional
Cycles	180,586,466	175,852,925,081	~974×
Instructions	251,989,811	230,822,559,530	~916×
IPC (insn/cycle)	1.39	1.31	~equal
Context-switches	16	34,257	~2141×
Page-faults	1,941	2,367,004	~1219×

Traditional wins by roughly three orders of magnitude on every metric that measures work done, while IPC, the metric that measures efficiency *per instruction*, is nearly identical between the two. FaaS isn't doing the same work less efficiently; it's doing far more total work for an identical result. That extra work is 1,205 separate Python interpreter cold starts (200 rides times 6 operations, plus 5 initial driver clock-ins), each re-importing modules and re-initializing the CPython runtime to run a handful of lines of logic and exit. The context-switch (~2141×) and page-fault (~1219×) numbers are the OS-level fingerprint of exactly that: every `subprocess.run()` is a `fork()+exec()`, meaning a new address space, freshly faulted-in page tables, and a new process for the scheduler to context-switch into. Dividing 78.3s by 1,205 operations gives ~65ms/operation, squarely in the range of a cold Python start, not of running a few lines of matching/fare logic.

FlameGraphs confirm this visually.

	Traditional	FaaS
Stack frames captured	136	5,627
Run duration sampled	~0.08 s (20,000 rides)	~22 s (50 rides)
Dominant cost	<code>dataclasses.asdict()</code> (~50% of frames)	<code><frozen importlib._bootstrap...></code> (~61% of frames)



Traditional's graph is dominated by the request/match logic, plus a real, minor hotspot in `dataclasses.asdict()`: roughly half of all sampled frames pass through it, a fixable inefficiency the profiling surfaced that we hadn't otherwise noticed.



FaaS's graph looks structurally different: hundreds of separate narrow towers, one per spawned subprocess, instead of one shared call tree. Roughly 61% of all captured frames sit inside Python's import machinery, meaning the majority of each operation's lifetime is spent starting the interpreter and importing modules, not executing handler().

This mirrors real cloud FaaS platforms, which fight this exact cold-start cost with warm execution pools. Our subprocess-per-call design has no such mechanism, deliberately, so the underlying cost is visible rather than hidden.

AI Tool Usage

We used Claude (Anthropic), via Claude Code, as a collaborator throughout this assignment: to discuss the architectural tradeoffs and pick a feature extension worth demonstrating, to help write the implementation and tests for both architectures, to debug issues that came up during testing and profiling, and to help interpret and write up the perf/flamegraph results in Part 4. We reviewed and tested everything before treating it as final (47 passing tests plus the cross-architecture equivalence checks), and the architectural conclusions in Parts 3 and 4 reflect our own analysis of the concrete, measured behavior of both implementations.

Conclusion

The two architectures agree on outputs but diverge sharply everywhere else. Traditional is faster by roughly three orders of magnitude, confirmed by both perf stat and the flamegraphs, and cheaper to extend for cross-cutting features. FaaS's per-call interpreter cold-start tax and its composition cost, which only appears once a feature needs more than one function, are the two concrete prices of the architecture's independent-function design in this implementation.